CPSC 481
Artificial Intelligence

Chuck Chekuri

cchekuri@fullerton.edu

LinkedIn

Group

Syllabus

So far…

- Week 1: History of Intelligence and Intelligence
- Week 2: History of AI and Types of AI
- Week 3: P.E.A.S. and Agent architectures
- Week 4: Agentic AI – Types, SPA, Goals, MAS.

Recap: Goal-Based Agents

- Agents need to know the "desirability" of states.
- Goals provide a target but not the "how."
- Problem-solving agents are a specific type of goal-based agent.
- They use atomic representations (states are indivisible wholes).
- Search is the process of finding the action sequence.

Week 5: Solving Problems by Searching

Transitioning from General Agents to Problem Solvers.

Core Focus: How agents look ahead to achieve goals.

Key Framework: Search algorithms in atomic environments.

Textbook Reading: AIMA 4th Ed. Chapter 3 (Sec 3.1-3.5).

Week's Goal: Formulating problems and uninformed search.

The "Search" Paradigm

- Search is used when the correct action is not immediate.
- It involves a "mental" exploration of possibilities.
- Search happens in the agent's "head," not in the world.
- Execution only begins after a solution path is found.
- This is a "Plan-then-Execute" (open-loop) approach.

Fig 3.1 from the textbook

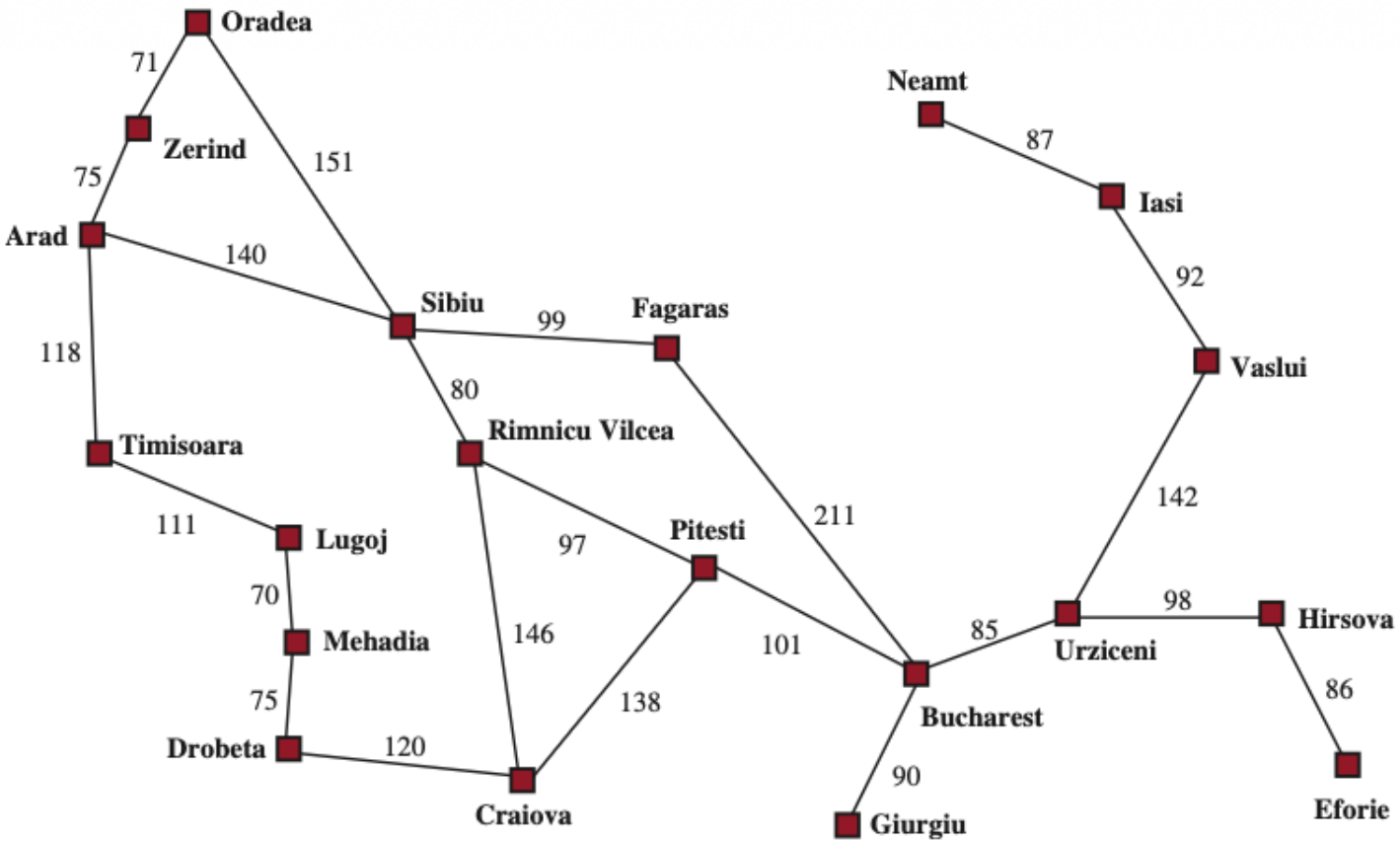

Figure 3.1 A simplified road map of part of Romania, with road distances in miles.

Section 3.1: Problem-Solving Agents

- Example: Vacationing in Romania (Current: Arad, Goal: Bucharest).
- The agent must find a sequence of cities to travel through.
- Goal Formulation: Limiting the world to the desired target.
- Problem Formulation: Defining the states and actions needed.
- The agent "looks ahead" before committing to a road.

The Simple Problem-Solving Agent Algorithm

- Input: A percept (current state). Output: An action.
- Step 1: Update internal state from percept.
- Step 2: If no plan exists, formulate a goal and a problem.
- Step 3a: Goal and a Search for a Solution (action sequence).
- Step 4: Execute the plan's actions one by one.

Assumptions in Problem-Solving Search

- Fully Observable: Agent always knows where it is.
- Deterministic: Actions have certain, known outcomes.
- Discrete: States and actions are countable and distinct.
- Static: The world doesn't change while the agent thinks.
- Known: The "physics" of the world are known to the agent.

Importance of Abstraction

- The world is too complex for 1:1 modeling.
- Abstraction: Removing irrelevant details (e.g., tire pressure).
- A "State" is an abstraction of the physical world.
- An "Action" is an abstraction of physical movement.
- Valid Abstractions: Ensure solutions work in the real world.

Atomic vs. Factored Representations

- Problem-Solving Agents use ATOMIC representations.
- States are "Black Boxes" (e.g., "Arad" or "Sibiu").
- No internal structure is visible to the search algorithm.
- Planning agents (later chapters) use structured states.
- Search algorithms only care about "equality" of states.

8-Puzzle

Start State

Goal State

Section 3.2: Well-Defined Problems

- A problem is defined by five components.
- 1. Initial State: Where the agent begins (e.g., In: Arad).
- 2. Actions(s): Returns the set of possible actions from state s.
- 3. Transition Model: Result(s, a) returns the outcome state.
- 4. Goal Test: Is the state a goal? (Explicit or Implicit).
- 5. Path Cost: Step-cost c(s, a, s') to move between states.

The Goal Test: Explicit vs. Implicit

- Explicit Goal: A specific state (e.g., "State = Bucharest").
- Implicit Goal: A property (e.g., "Checkmate" in Chess).
- The Goal Test is a Boolean function applied to a state.
- Efficient goal tests are vital for search performance.
- The agent stops once the goal test returns TRUE.

Path Cost and Step Cost

- Path Cost (g): Sum of costs along the action sequence.
- Step Cost: Cost to perform action 'a' in state 's' to reach 's_prime'.
- Costs represent time, distance, fuel, or money.
- Optimal Solution: The path with the lowest path cost.
- Most search algorithms assume step costs are non-negative.

The State Space

- The set of all states reachable from the initial state.
- Often represented as a directed graph.
- Nodes = States; Edges = Actions.
- A "path" in the state space is a sequence of states.
- Complexity is determined by the size of the state space.

Search Tree vs. State Space Graph

- State Space: Graph showing all possible connections.
- Search Tree: Explores paths from the root (initial state).
- A state in the graph may appear as many nodes in the tree.
- Tree nodes include parent pointers to reconstruct the path.
- Redundant paths can make trees infinite even if graphs are finite.

Insight

Chapter 2 (Environment)	Chapter 3 (Search Structure)
Deterministic	Single successor per action
Stochastic	Probabilistic branching
Fully observable	Concrete state representation
Partially observable	Belief-state search
Discrete	Graph search
Continuous	Infinite state space
Sequential	Tree expansion
Multi-agent	Game tree

Example: The 8-Puzzle

- States: Locations of the 8 tiles and the blank space.
- Initial State: Any configuration.
- Actions: Move the blank Up, Down, Left, or Right.
- Goal Test: Match a specific pattern of tiles.
- Path Cost: Each move costs 1 (total = number of moves).

Complexity of the 8-Puzzle

- State space size: 9! / 2 = 181,440 reachable states.
- The 15-puzzle has 10^13 states.
- This is an NP-complete problem.
- Illustrates the need for efficient search strategies.
- Even "toy" problems can have massive state spaces.

Example: The 8-Queens Problem

- Goal: Place 8 queens on a board so no two attack each other.
- Incremental Formulation: Start with 0 queens, add 1 per step.
- States: Any arrangement of 0-8 queens on the board.
- Action: Place a queen in an empty square.
- Complete-State Formulation: Start with 8 queens, move them.

Real-World Problems: Route Finding

- Beyond Romania: Map-based navigation (Google Maps).
- States: Locations (Latitude/Longitude).
- Actions: Road segments between intersections.
- Path Cost: Time, distance, or "scenic value."
- Used in airlines, military planning, and internet routing.

Real-World Problems: VLSI Layout

- Goal: Positioning millions of components on a chip.
- Wires must not cross; area must be minimized.
- This is a massive search problem.
- Split into "Cell Placement" and "Routing."
- Requires specialized, high-performance search algorithms.

Section 3.3: Search Algorithms

- Core Idea: Expanding states to generate a search tree.
- "Frontier" (Open List): Set of nodes waiting to be expanded.
- Expanding: Applying Actions(s) to generate successor nodes.
- "Reached" (Closed List): States already visited.
- Strategy: Determines which node from the frontier to expand next.

Node Data Structure

- A Node is DIFFERENT from a State.
- Node.STATE: The state it represents.
- Node.PARENT: The node that generated this one.
- Node.ACTION: The action applied to the parent.
- Node.PATH-COST: Total cost g(n) from the root.

The Best-First Search Template

- All search strategies follow a similar template.
- Place the initial state in the frontier.
- Loop: If frontier is empty, return failure.
- Choose node 'n' from the frontier using an evaluation function.
- If Goal-Test(n) is true, return the solution.
- Expand 'n' and add successors to the frontier.

Handling Redundant Paths

- Cycle: A path that returns to a previously visited state.
- Redundant Path: Multiple paths to the same state.
- These can lead to infinite loops in search trees.
- Solution: Maintain a "Reached" set of visited states.
- If a newly generated state is in "Reached," discard it.

Graph-Search vs. Tree-Search

- Tree-Search: Does not track "Reached" states (simple but risky).
- Graph-Search: Explicitly tracks "Reached" set.
- Graph-Search is mandatory for most real-world problems.
- Tradeoff: Graph-Search saves time but uses more memory.
- Most modern algorithms imply Graph-Search.

Measuring Search Performance

- Completeness: Does it find a solution if one exists?
- Optimality: Does it find the LOWEST cost solution?
- Time Complexity: How long does it take (nodes expanded)?
- Space Complexity: How much memory (nodes in frontier/reached)?

Complexity Variables: b, d, m

- b: Branching factor (max successors of any node).
- d: Depth of the shallowest goal node.
- m: Maximum length of any path in the state space.
- Time is often O(b^d).
- Space is often the bottleneck in search algorithms.

Section 3.4: Uninformed Search Strategies

- Also known as "Blind Search."
- Agent has no "hint" about how far a goal is.
- Can only distinguish between goal and non-goal states.
- Strategies differ by the order of node expansion.
- Key Algorithms: BFS, Dijkstra, DFS, IDS, Bidirectional.

Breadth-First Search (BFS)

- Strategy: Expand the shallowest unexpanded node.
- Implementation: Frontier is a FIFO queue.
- New nodes go to the back; expansion takes from the front.
- Explores the tree level by level.
- Completeness: Yes (if b is finite).

BFS: Optimality and Complexity

- Optimality: Yes, ONLY if all step costs are equal.
- Time Complexity: O(b^d).
- Space Complexity: O(b^d) - all nodes are kept in memory.
- Memory is the biggest problem for BFS.
- A depth of 10 might require gigabytes of RAM.

Dijkstra's Algorithm (Uniform-Cost Search)

- BFS is not optimal when step costs vary.
- Strategy: Expand node 'n' with lowest path cost g(n).
- Implementation: Frontier is a Priority Queue (ordered by g).
- Goal test is applied when node is EXPANDED, not generated.
- Reaches the goal via the cheapest path.

Why Goal Test at Expansion?

- In Uniform-Cost, the first goal GENERATED might not be the cheapest.
- We must wait until the goal is the cheapest in the frontier.
- Example: Path A->B->C (cost 10) vs A->D (cost 20).
- If D is the goal, we find it only after exploring B and C.
- Ensures cost-optimality for any non-negative step costs.

Depth-First Search (DFS)

- Strategy: Expand the deepest unexpanded node.
- Implementation: Frontier is a LIFO queue (Stack).
- Goes down one branch until it hits a leaf or limit.
- Space Complexity: O(bm) - only stores path to current node.
- Much more memory-efficient than BFS.

DFS: The Drawbacks

- Completeness: No (can get stuck in infinite paths).
- Optimality: No (might find a deep goal before a shallow one).
- Time Complexity: O(b^m).
- Risky in state spaces with cycles or infinite depth.
- Often used with a "Depth Limit" to prevent getting lost.

Depth-Limited Search

- DFS with a predetermined depth limit 'l'.
- Nodes at depth 'l' are treated as having no successors.
- Solves the infinite path problem.
- Problem: How to choose 'l'?
- If $l < d$, the goal is unreachable.

Iterative Deepening Search (IDS)

- Strategy: Repeatedly apply Depth-Limited Search.
- Starts with limit 0, then 1, then 2... until goal found.
- Combines the benefits of BFS and DFS.
- Memory: O(bd) like DFS.
- Completeness/Optimality: Like BFS (if costs equal).

Is IDS Wasteful?

- It seems wasteful to repeat the top of the tree.
- However, most nodes are at the bottom level.
- Example b=10: Bottom level has 1,000,000; total tree has 1,111,111.
- Top levels are small; re-exploring them has little impact.
- IDS is the preferred uninformed search for large state spaces.

Bidirectional Search

- Strategy: Search forward from start AND backward from goal.
- Stops when the two frontiers meet.
- Complexity: O(b^(d/2) + b^(d/2)), which is much smaller than O(b^d).
- Problem: Must be able to search backward from the goal.
- Requires multiple goals to be well-defined.

Summary of Uninformed Strategies

- BFS: Optimal for unit cost; massive memory usage.
- Uniform-Cost: Optimal for any non-negative cost.
- DFS: Memory efficient; not complete or optimal.
- IDS: Best overall uninformed choice for large spaces.
- Bidirectional: Very fast if backward search is possible.

Section 3.5: Informed (Heuristic) Search

- Blind search expands nodes in all directions.
- Informed search uses "Hints" about the goal's location.
- Heuristic Function h(n): Estimated cost from node n to goal.
- Example: Straight-line distance to Bucharest.
- Informed search finds solutions much more efficiently.

Comparing Search Algorithms (Table)

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes[1]	Yes[1,2]	No	No	Yes[1]	Yes[1,4]
Optimal cost?	Yes[3]	Yes	No	No	Yes[3]	Yes[3,4]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Figure 3.15 Evaluation of search algorithms. b is the branching factor; m is the maximum depth of the search tree; d is the depth of the shallowest solution, or is m when there is no solution; ℓ is the depth limit. Superscript caveats are as follows: [1] complete if b is finite, and the state space either has a solution or is finite. [2] complete if all action costs are $\geq \epsilon > 0$; [3] cost-optimal if action costs are all identical; [4] if both directions are breadth-first or uniform-cost.

Week 5 Summary & Looking Ahead

- Search is a primary tool for problem-solving agents.
- Problem formulation is the first step in AI design.
- Uninformed search is the baseline for all AI planning.
- Next Week: Informed Search (A* and Heuristics).
- Assignment: Implement BFS and DFS on the Romania map.

Sample Question 1

Rationality & Performance Measures

Choosing Between Two Job Offers: Taylor has received two job offers:

- **Offer A:** High salary, demanding workload, limited mentorship, fast promotion potential.
- **Offer B:** Moderate salary, strong mentorship, slower promotion, better work-life balance.

Taylor must choose one. Future market conditions and burnout risk are uncertain.

Your Task: Write three paragraphs:

1. **Paragraph 1 (≤150 words)**
   Define Taylor's performance measure P. What is being maximized?
2. **Paragraph 2 (≤200 words)**
   State the constraints or risk profile you are assuming.
3. **Paragraph 3 (≤250 words)**
   Explain why your selected offer maximizes Punder those assumptions.

Sample Question 2

Problem Formulation Shapes Difficulty

Managing Weekly Study Time
Alex is taking five demanding courses.
Each week, Alex must allocate a limited number of study hours across them.
The problem could be formulated as:

- Allocating all hours at once
- Sequential weekly adjustments
- Satisfying minimum grade thresholds
- Maximizing total semester GPA

Alex has limited computational resources.

Your Task: Write three paragraphs:

1. **Paragraph 1 (≤150 words)**
   Define your proposed state representation and actions.
2. **Paragraph 2 (≤200 words)**
   Explain how your representation affects the structure of the state space.
3. **Paragraph 3 (≤250 words)**
   Explain why your formulation is preferable under your assumptions.

Sample Question 3

Search Guarantees Are Conditional

Searching for an Apartment

Morgan is searching for an apartment near campus.

Each apartment requires:

- An application fee
- A background check
- Time to visit

Different apartments have different costs and response times.

Morgan wants to minimize total time and money before securing a lease.

Your Task: Write three paragraphs:

1. **Paragraph 1 (≤150 words)**
   Define the search strategy you propose.
2. **Paragraph 2 (≤200 words)**
   State your assumptions about costs, timing, and availability.
3. **Paragraph 3 (≤250 words)**
   Explain why your strategy provides appropriate guarantees and tradeoffs under those assumptions.

Sample Question 4

Heuristics: Efficiency vs. Optimality

Prioritizing Graduate School Applications

Riley is applying to graduate schools.

Riley uses a heuristic score based on:

- Program ranking
- Faculty alignment
- Location
- Acceptance rates

The heuristic is imperfect.

Your Task: Write three paragraphs:

1. **Paragraph 1 (≤150 words)**
   Define the heuristic function.
2. **Paragraph 2 (≤200 words)**
   State assumptions about its reliability.
3. **Paragraph 3 (≤250 words)**
   Explain how relying on this heuristic affects efficiency and optimality.